Report week 2

Project: Construction of Judgement Models from Imperfect Human Expert Feedback

Track: Research

Problem statement

In educational institutions, a significant portion of mentors' time is spent on the routine grading of student work. Existing approaches to automating feedback using large language models (LLMs) suffer from insufficient reliability and highly variable results. At the same time, the expert knowledge used to evaluate solutions is typically not formalized and exists only as an implicit mental model held by the expert.

It is necessary to develop an approach that allows for the extraction and gradual refinement of the expert's evaluation model based on a stream of natural-language feedback.

Justification

The project meets the Research Track acceptance criteria.

Clear and Testable Problem

The project addresses the problem of automatically constructing and refining judgement models from imperfect human expert feedback. The problem is specific, measurable, and testable through benchmark evaluation metrics such as False Positive Rate (FPR) and False Negative Rate (FNR).

Concrete Deliverables

The project will produce several tangible outcomes, including:

- a judgement model extraction and update pipeline;

- an evaluation-model diff generator;
- a criterion vector conversion mechanism;
- integration with the existing benchmark framework;
- an experimental FastAPI and Streamlit application;
- benchmark results and research findings.

Feasible Scope

The project is appropriately scoped for a six-week research project. It builds upon an existing dataset, benchmark framework, and predefined project requirements, allowing the team to focus on experimentation and evaluation rather than building all components from scratch.

Measurable Success Criteria

Success will be evaluated through benchmark metrics, including FPR, FNR, generation cost, generation latency, and model convergence toward the reference evaluation model.

Research Question

Can an evaluation model automatically constructed from imperfect expert feedback converge toward a reference evaluation model and accurately identify violated criteria in unseen solutions?

Research Gap

Existing approaches generally assume that evaluation criteria already exist in an explicit form or can be manually formalized by experts. In practice, expert judgement often remains implicit and embedded within expert intuition. Limited research has explored whether such judgement models can be automatically reconstructed, maintained, and refined from streams of imperfect natural-language feedback. This project investigates whether an explicit judgement model can emerge and converge toward a reference model through iterative processing of mentor feedback.

Research Hypothesis

If expert comments are converted into a structured model of binary evaluation criteria, then the resulting judgement model will gradually improve as more feedback is incorporated, leading to lower FPR and FNR values compared to direct feedback generation approaches.

Research Contribution

The project is not limited to tool development. It includes investigation of judgement model construction, model convergence, criterion extraction strategies, vector conversion approaches, and comparative evaluation of different LLM-based architectures through systematic benchmark experiments.

Ethical Considerations

The project operates on educational assessment data and focuses on supporting human evaluators. Human verification remains part of the evaluation process, and fully autonomous decision-making is explicitly outside the project scope.

Team

Name	Innopolis University email	Role in the team
Alena Petrenko	al.petrenko@innopolis.university	Team leader, researcher
Elizaveta Bubnova	e.bubnova@innopolis.university	Reporter
Amaliya Kharisova	a.kharisova@innopolis.university	Researcher
Victoria Yurina	v.yurina@innopolis.university	App developer
Polina Sych	p.sych@innopolis.university	App developer
Arina Nikolaeva	a.nikolaeva@innopolis.university	Researcher

Project plan & Milestones

Week 2

- App developer 1: set up FastAPI backend skeleton (routes, project structure), commit initial repo
- App developer 2: set up Streamlit frontend skeleton, define API contract between frontend and backend
- Researcher 1: design the evaluation model JSON schema
- Researcher 2: design the binary criterion vector format and define the criteria extraction pipeline architecture (feedback to model)
- Researcher 3: create an initial model
- Reporter: set up reporting template, commit Week 2 report

Week 3

- App developer 1: continue developing FastAPI backend skeleton
- App developer 2: continue developing Streamlit frontend skeleton
- Researcher 1: start researching binary criterion vector conversion rules (A to B rules from the spec)
- Researcher 2: start researching binary criterion vector conversion rules (A to B rules from the spec)
- Researcher 3: describe the integration contract with the existing feedback quality benchmark
- Reporter: commit Week 3 report

Week 4

- Researcher 1: look at the data of the previous runs
- Researcher 2: create a new hypothesis
- Researcher 3: adjust the approach
- App developer 1: update the pipeline according to a new approach
- App developer 2: update the pipeline according to a new approach
- Reporter: commit Week 4 report

Week 5

- Researcher 1: look at the data of the previous runs
- Researcher 2: create a new hypothesis
- Researcher 3: adjust the approach
- App developer 1: update the pipeline according to a new approach
- App developer 2: update the pipeline according to a new approach
- Reporter: commit Week 5 report

Week 6

- Researcher 1: look at the data of the previous runs
- Researcher 2: create a new hypothesis
- Researcher 3: adjust the approach
- App developer 1: update the pipeline according to a new approach
- App developer 2: update the pipeline according to a new approach
- Reporter: commit Week 6 report

Risks

1. **Insufficient data for training and evaluating the model.**

   The project begins with a small initial dataset (3 cases and 9 solutions), but conducting comprehensive experiments requires an expanded dataset of more than 75 solutions with a uniform distribution of errors.

2. **Low quality of evaluation model extraction from text-based feedback.**

   Mentor feedback is a loosely formalized source of information, so the extracted criteria may incompletely or inaccurately reflect the actual expert evaluation model.

3. **High variability in LLM results.**

   The non-deterministic nature of language models can lead to unstable extraction of criteria and variations in evaluation results across runs.

4. **Ambiguity in mapping generated criteria to the reference evaluation model.**

   The generated evaluation model may contain criteria that do not directly correspond to the criteria of the reference model. A single generated criterion may map to multiple reference criteria, while criterion ordering and naming can vary across model versions. This may complicate vector conversion and negatively affect benchmark results.

5. **Failure to achieve the required quality level of the evaluation model.**

   The proposed approach may fail to reduce the FPR and FNR metrics to the expected values even after several model update iterations.

6. **High cost or execution time of experiments.**

   A large number of LLM calls may exceed acceptable costs or processing time, making it difficult to conduct the required number of experiments.

7. **Difficulty in constructing the extended dataset.**

   The project requires an extended dataset containing more than 75 solutions with evenly distributed error categories. Creating such a dataset while maintaining consistency of annotations and balanced error coverage may require significant effort and could delay experimentation.

Link to Kanban board

https://gitlab.pg.innopolis.university/al.petrenko/capstone-judgement-models/-/boards

Research Requirements

Research Questions

- Is it possible to automatically construct an evaluation model from imperfect human expert feedback?
- Can the extracted evaluation model be used to generate feedback with acceptable quality?
- Which architecture and base LLM provide the fastest convergence of the evaluation model toward the reference model?

Hypotheses

H1: If expert feedback is converted into a structured evaluation model consisting of binary verifiable criteria, then the quality of criterion prediction will improve over time as more feedback is incorporated into the model.

H2: A continuously updated evaluation model will achieve lower FPR and FNR values than a model generated from a limited amount of feedback.

H3: Different LLMs will demonstrate different effectiveness in extracting evaluation criteria and updating the evaluation model.

Methodology

The proposed methodology consists of the following stages:

1. Collect expert feedback for student solutions.
2. Extract evaluation criteria from the feedback using an LLM.
3. Compare the feedback against the current evaluation model and generate model updates by adding, refining, or merging criteria.
4. Represent the updated criteria as a structured JSON evaluation model.
5. Generate binary criterion predictions for new solutions using the current evaluation model.
6. Convert evaluation results into a binary criterion vector.
7. Convert the intermediate criterion vector into the format of the reference evaluation model.
8. Evaluate the generated model using the existing feedback-quality benchmark.
9. Update the evaluation model when new feedback becomes available.
10. Analyze the evolution of quality metrics across multiple iterations.

Dataset Plan

The project starts with an **initial dataset** consisting of:

- 3 tasks (cases);

- a reference evaluation model for each task;
- 9 annotated solutions in total;
- one correct solution and multiple erroneous solutions per task;
- expert feedback for each solution;
- case profiles represented as JSON evaluation models;
- conceptual blocks and binary evaluation criteria;
- annotated solutions stored as Jupyter notebooks;
- metadata containing violated criteria for each solution;
- ground-truth binary vectors derived from violated criteria.

The erroneous solutions are designed to cover different categories of mistakes and provide diverse feedback examples for evaluation model construction.

The dataset will be used for:

- evaluation model construction;
- criterion extraction experiments;
- benchmark evaluation;
- comparison of different LLM configurations.

Evaluation Plan

Baseline

The provided feedback-quality benchmark will be used as the baseline evaluation framework.

Before evaluating model convergence, the initial benchmark performance (base rate) will be measured. This baseline will be used to compare subsequent iterations of the evaluation model and assess the effectiveness of model updates over time.

Primary metrics:

- False Positive Rate (FPR);

- False Negative Rate (FNR).

Secondary metrics:

- feedback generation cost;
- feedback generation time;
- flip rate;
- statistical stability indicators.

The benchmark compares predicted binary criterion vectors against ground-truth vectors derived from dataset annotations. Evaluation results will be aggregated at criterion, block, case, and global levels.

Success criteria

The project aims to demonstrate that the evaluation model converges toward the reference model over multiple feedback iterations, resulting in decreasing FPR and FNR values while maintaining acceptable cost and generation time.

Related Work / Gap Analysis

Recent advances in Large Language Models (LLMs) have led to the development of numerous approaches for automated assessment and feedback generation in educational settings. Existing systems typically rely on either predefined rubrics created by experts or direct feedback generation using LLMs. These approaches can reduce the workload of mentors, but often suffer from inconsistency, limited transparency, and difficulty in maintaining evaluation standards across multiple iterations.

Rubric-based assessment methods provide structured and interpretable evaluation criteria, but they require experts to manually define and maintain the evaluation model. Creating and updating such rubrics is time-consuming and does not scale well when evaluation requirements evolve over time.

Recent LLM-based assessment approaches often generate feedback directly from student solutions without maintaining an explicit intermediate representation of expert knowledge. While these methods can produce useful feedback, they may exhibit variability across runs and provide limited insight into how evaluation decisions are made.

The proposed project investigates an alternative approach based on the automatic construction and iterative refinement of an explicit judgement model derived from accumulated expert feedback. Instead of relying solely on direct feedback generation, the system maintains a structured representation of evaluation criteria and continuously updates it as new feedback becomes available.

Therefore, the research gap addressed by this project is the limited exploration of methods for automatically constructing, maintaining, and evaluating explicit judgement models from imperfect human expert feedback. The project aims to determine whether such models can improve evaluation consistency and gradually converge toward a reference expert model.

Chosen Tech Stack

Technology	Purpose	Justification
Python	Core development language	Python is widely used in machine learning, natural language processing, and research projects. It provides extensive support for working with datasets, evaluation pipelines, and LLM integration.
Large Language Models (LLMs)	Criteria extraction and evaluation model generation	LLMs are the core component of the proposed approach. They are used to extract evaluation criteria from expert feedback, update evaluation models, and generate criterion predictions.
JSON	Evaluation model and dataset representation	The dataset stores case profiles, conceptual blocks, criteria, and metadata in JSON format. JSON is also used for storing generated evaluation models and experiment results.

Technology	Purpose	Justification
Jupyter Notebook	Solution representation and experimentation	Student solutions are provided as Jupyter notebooks (`.ipynb`). These notebooks serve as the primary input for evaluation and experimentation.
GitLab	Version control and project collaboration	GitLab is used for source code management, issue tracking, milestone planning,merge requests, and collaborative development.
FastAPI	Backend API	FastAPI is planned for implementing the backend services responsible for evaluation model generation, updates, and benchmark integration.
Streamlit	User interface	Streamlit is planned for building a lightweight web interface for running experiments and visualizing evaluation results.
Feedback Quality Benchmark	Evaluation framework	The provided benchmark framework is used to compare predicted criterion vectors against ground-truth vectors and calculate quality metrics such as FPR and FNR.
AITunnel	Access to LLM providers	Provides access to multiple LLM models used for evaluation model extraction, updating, and comparative experiments.

The selected technology stack is designed to support iterative construction of evaluation models from expert feedback. JSON-based datasets and evaluation models provide a structured representation of criteria and annotations. Jupyter notebooks serve as solution artifacts, while LLMs perform criterion extraction and model updates. FastAPI and Streamlit enable implementation of the experimental platform, and the benchmark framework provides objective evaluation through quality metrics such as FPR and FNR. GitLab supports collaborative development and experiment reproducibility throughout the project lifecycle.

MVP Description

At the current stage, the project team has established the initial application architecture and development environment required for future experimentation.

Backend Implementation

The backend skeleton has been implemented using FastAPI. The current implementation already exposes initial API endpoints for experiment execution, feedback submission, model retrieval, and health monitoring. API documentation is automatically generated using Swagger/OpenAPI. The backend provides the foundation for future integration of the judgement model construction pipeline, benchmark execution, and interaction with external LLM APIs. The current implementation defines the project structure, API architecture, and the interfaces required for communication with other system components.

Frontend Implementation

A functional Streamlit prototype has been implemented. The application currently provides navigation between experiment and model management modules, visualization of benchmark progress, experiment execution controls, task specification management, and basic user interaction components. The interface serves as a foundation for future integration with the backend experimentation pipeline.

Dataset Structure

An initial dataset structure has been defined and documented. Each case is represented as a JSON evaluation model containing metadata, conceptual blocks, and binary evaluation criteria. The structure provides a formal representation of evaluation requirements and defines the ground-truth criterion space for benchmark evaluation.

Ground Truth Representation

A standardized annotation format has been specified through solution metadata. Each solution is associated with a list of violated criteria, allowing automatic construction of ground-truth binary vectors used for evaluation.

Benchmark Preparation

The benchmark input and output formats have been analyzed and documented. The benchmark is designed to compare predicted criterion vectors against ground-truth vectors and calculate evaluation metrics, including False Positive Rate (FPR), False Negative Rate (FNR), generation cost, generation latency, and flip rate.

Experiment Workflow Design

The end-to-end experimentation workflow has been designed and documented. The workflow defines how mentor-style feedback is used to iteratively improve the evaluation model and how benchmark metrics are used to measure convergence toward the reference model. At this stage, the workflow has been designed and documented conceptually. Full implementation of the experiment management pipeline is planned for subsequent project iterations.

Evaluation Pipeline Design

The experimental process separates two parallel flows:

- a feedback-processing flow responsible for updating evaluation criteria based on natural-language feedback;
- a benchmark-evaluation flow responsible for measuring model quality through binary-vector comparison and benchmark metrics.

This separation provides a clear distinction between model construction and model evaluation.

The converter and diff generator components are currently defined at the architectural level and will be implemented in future iterations.

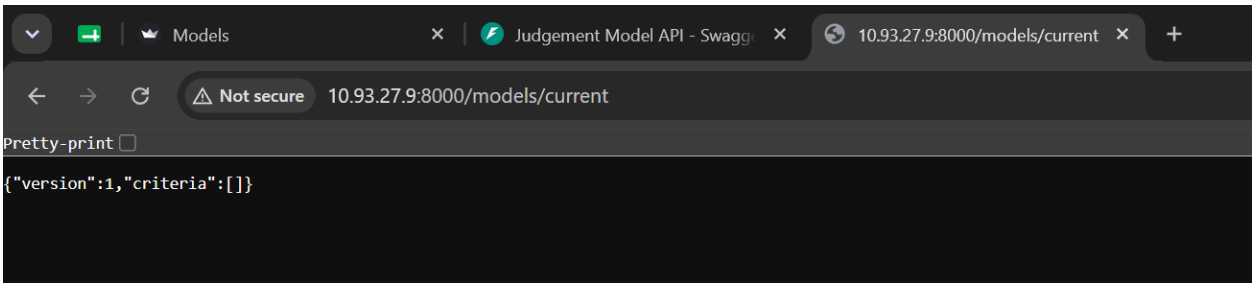
Iterative Model Improvement Process

The project defines a cyclic evaluation process in which feedback generated for a solution is used to update the current evaluation model. The updated model becomes the starting point for the next iteration, allowing gradual refinement and convergence toward the reference expert model.

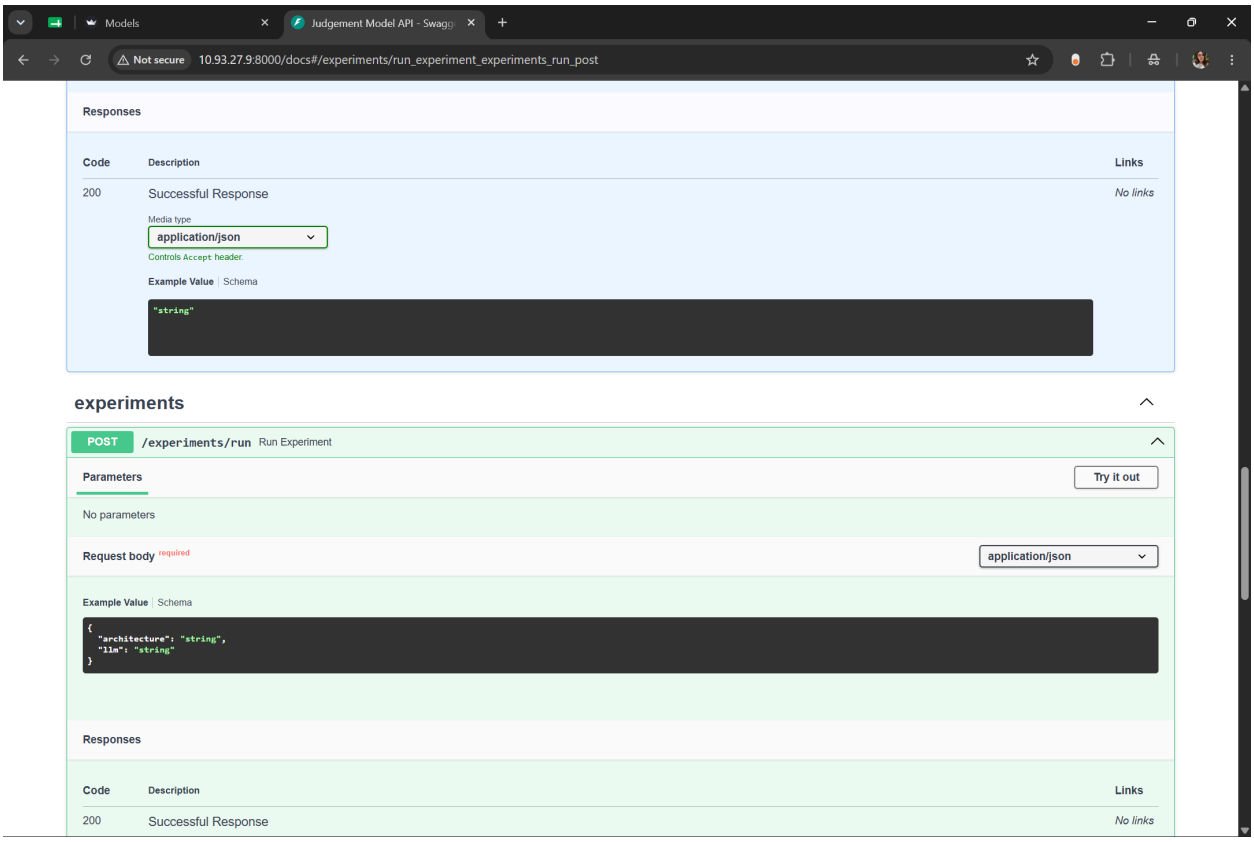
The current work focuses on defining and validating this iterative process before implementing the complete automation pipeline.

MVP Demonstration

Backend API

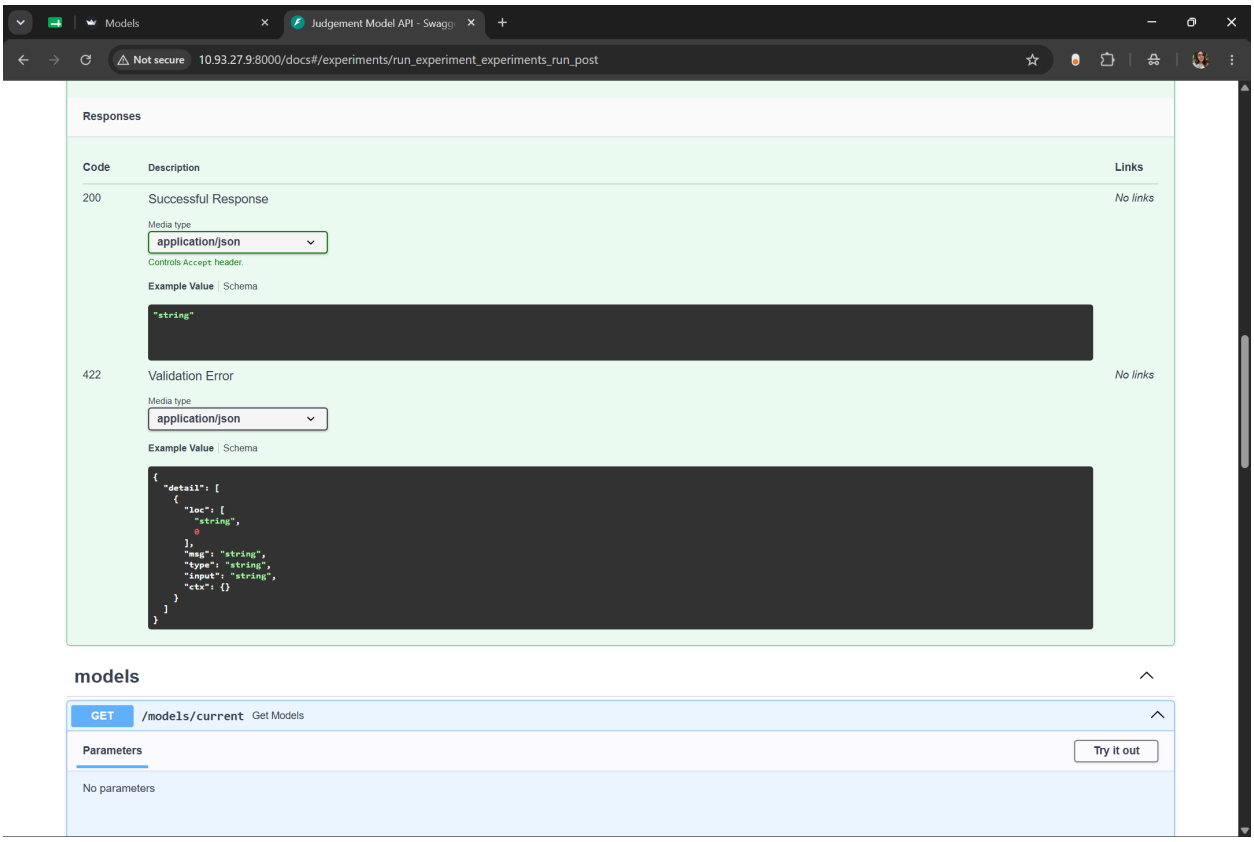


Current model endpoint response (`/models/current`)

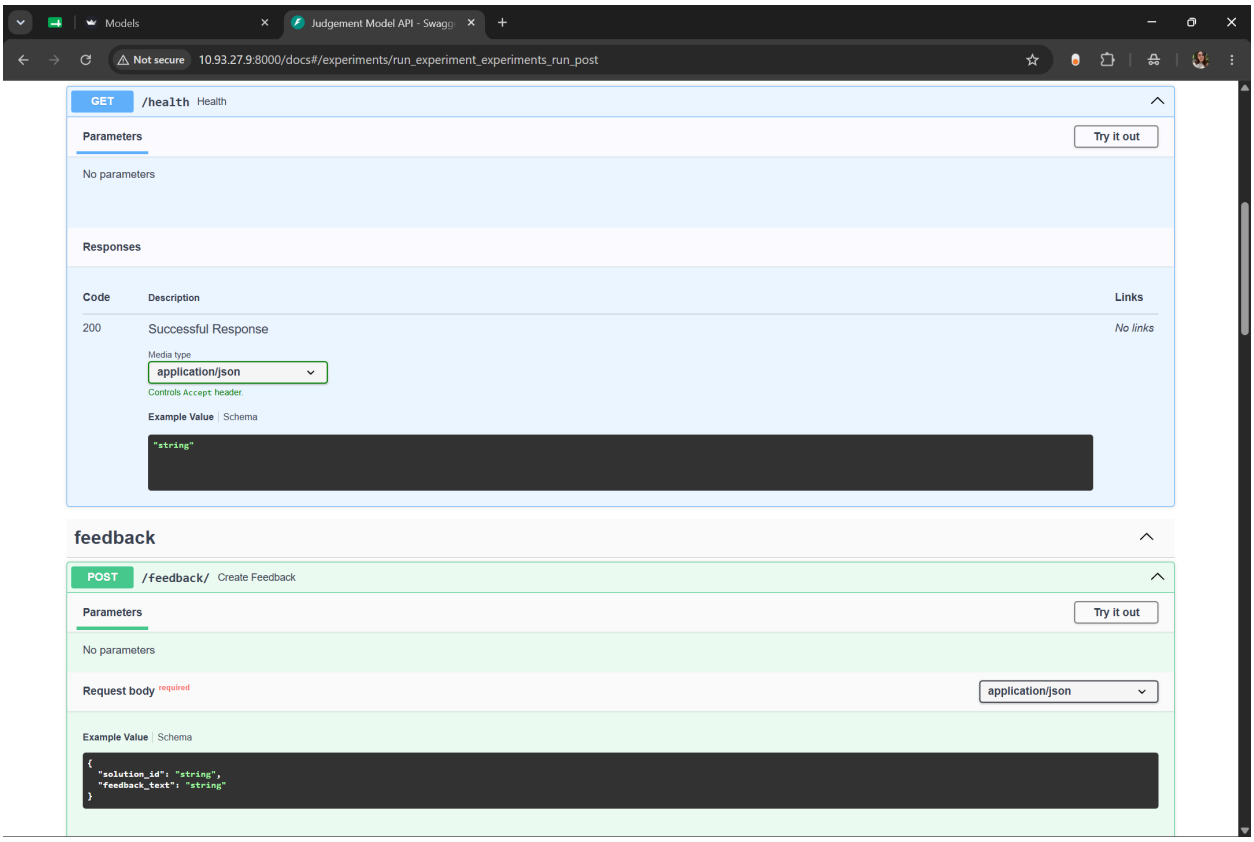


Swagger documentation for the `/experiments/run` endpoint

The implemented FastAPI backend exposes endpoints for experiment execution, feedback submission, model retrieval, and health monitoring. Automatic OpenAPI documentation is generated through Swagger UI.

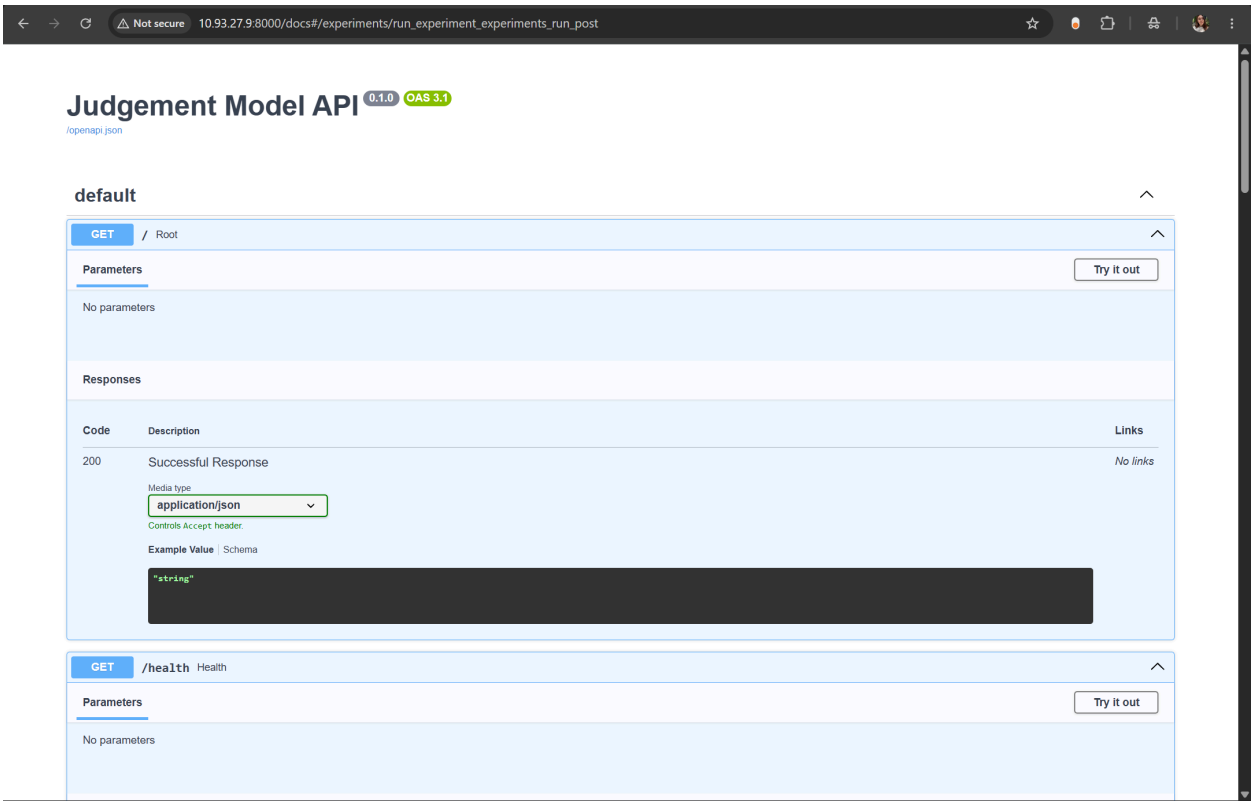


Current model endpoint (`GET /models/current`)



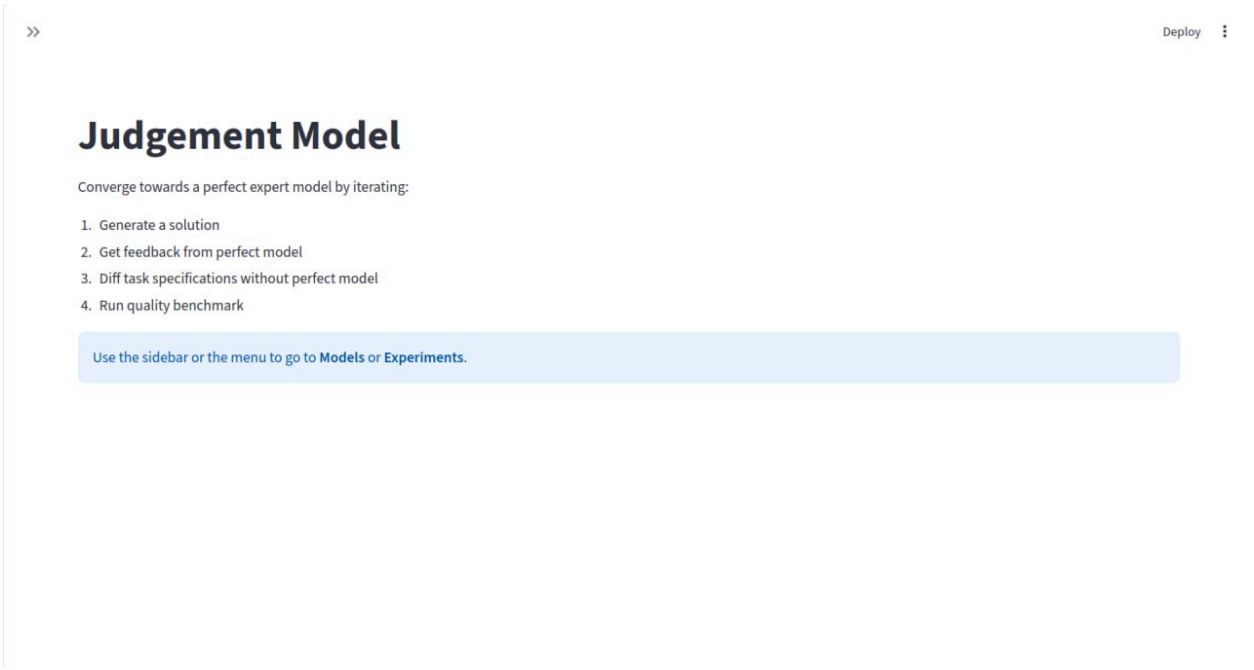
Feedback submission endpoint (`POST /feedback/`)

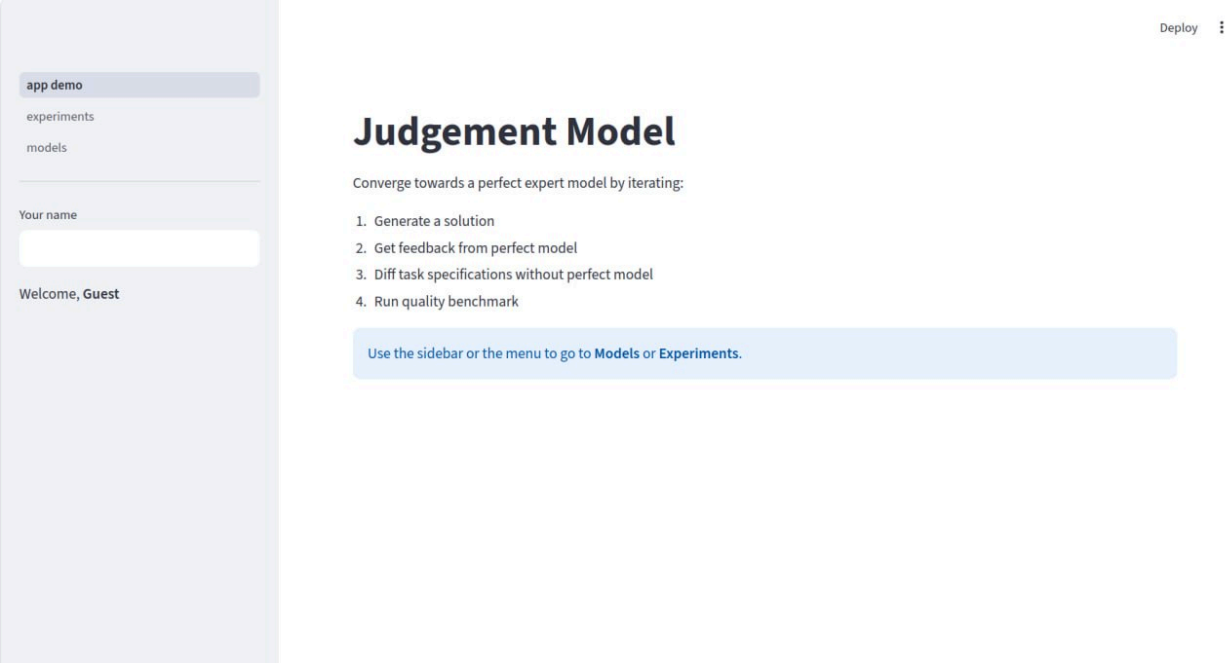
Current Model Endpoint



Swagger API Overview

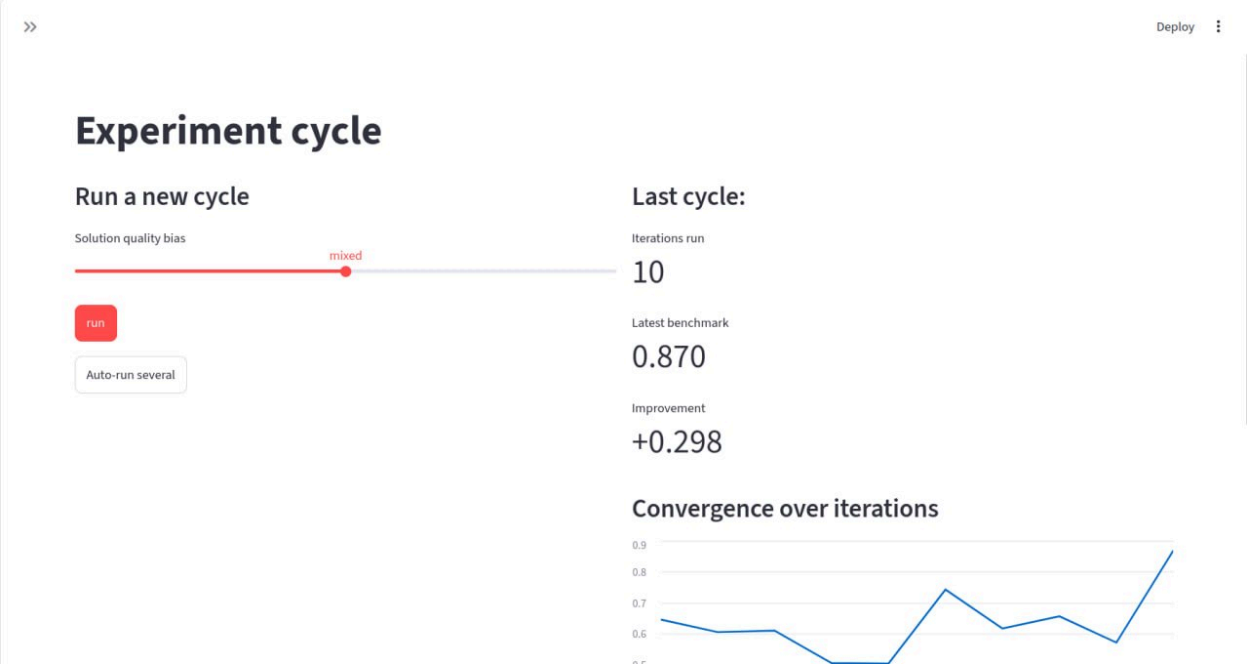
Home Page





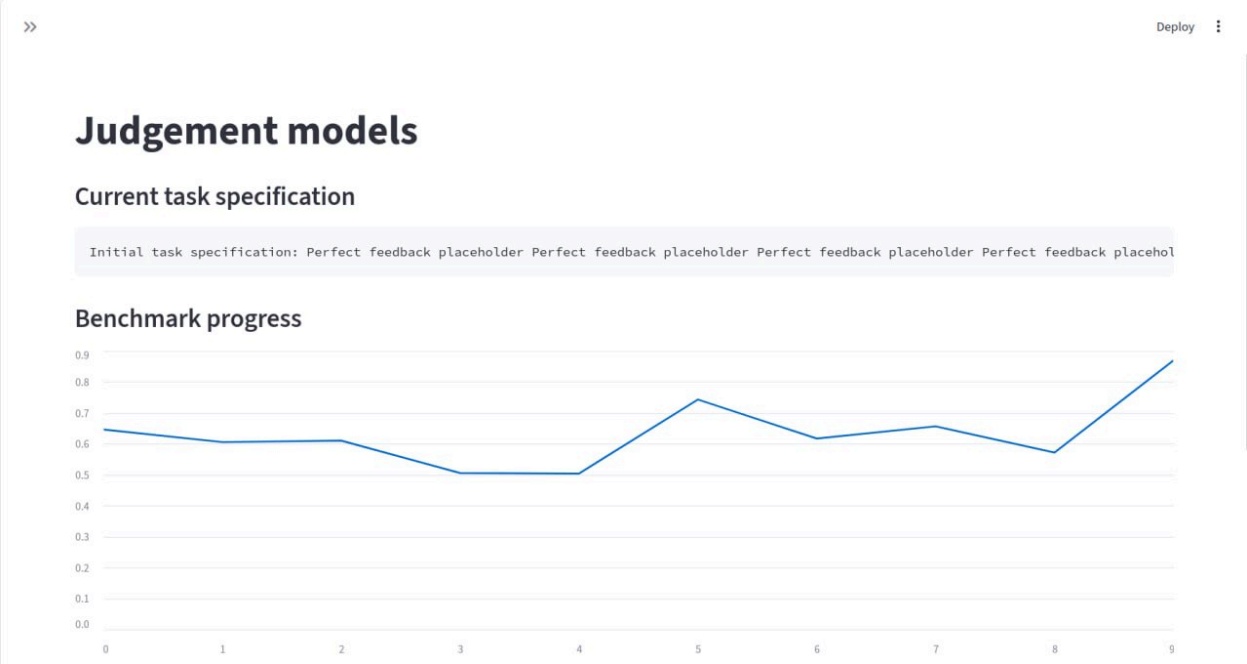
Main application page. The interface introduces the judgement model construction workflow and provides navigation to experiment and model management modules.

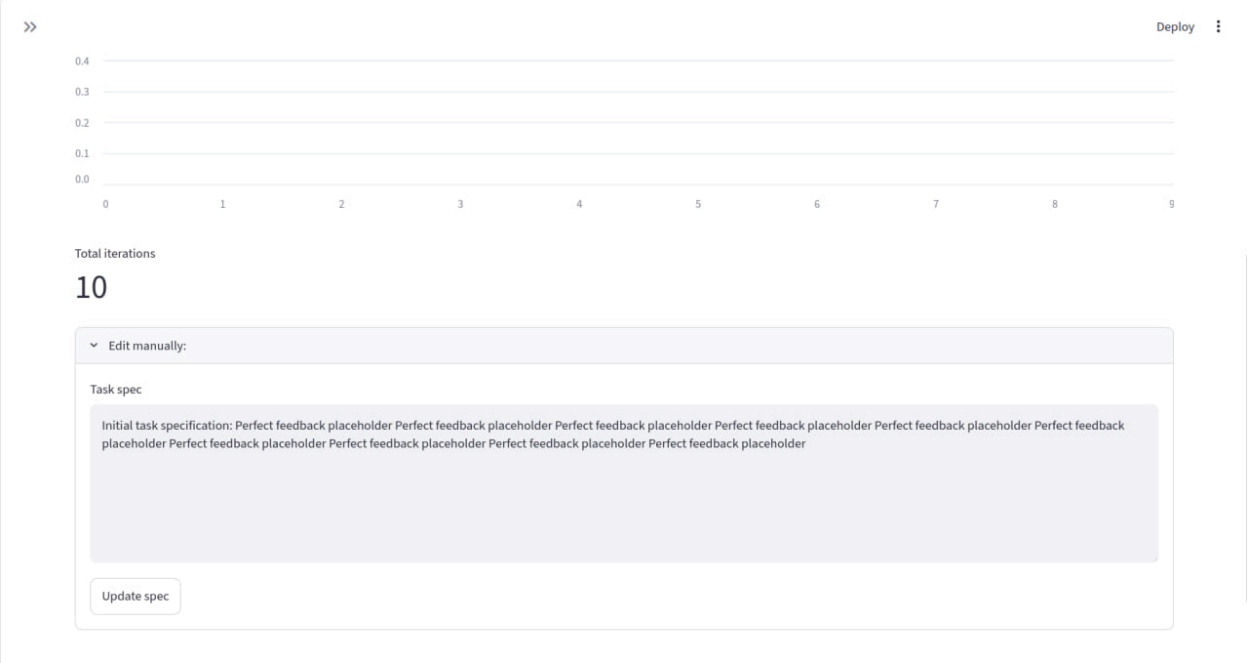
Experiment Management Interface



Experiment execution interface. Researchers can launch experiment cycles, monitor benchmark scores, and observe convergence trends across iterations.

Model Visualization Interface





Evaluation model management interface. The application displays the current task specification, benchmark history, and provides facilities for manual model updates.

Initial Baseline Configuration

The first benchmark scenario has been prepared using a reference evaluation model containing three binary criteria:

- Load housing price dataset from CSV into a pandas DataFrame.
- Verify the absence of missing values after handling.
- Display the number or proportion of price outliers.

These criteria define the initial ground-truth criterion space used for the first benchmark experiments. The corresponding criterion vectors will serve as the baseline reference for evaluating generated judgement models and measuring future convergence through FPR and FNR metrics.

Initial Evaluation Model Update Prototype

A prototype implementation of evaluation model updating has been created. The prototype sends feedback and the current evaluation model to multiple LLMs

through AITunnel and generates updated JSON-based evaluation models. The outputs of different models are stored for further comparative analysis and architecture evaluation.

Relevant Issues

Issue 1:

https://gitlab.pg.innopolis.university/al.petrenko/capstone-judgement-models/-/issues/1

Issue 2:

https://gitlab.pg.innopolis.university/al.petrenko/capstone-judgement-models/-/issues/2

Issue 3:

https://gitlab.pg.innopolis.university/al.petrenko/capstone-judgement-models/-/issues/3

Issue 4:

https://gitlab.pg.innopolis.university/al.petrenko/capstone-judgement-models/-/issues/4

Issue 5:

https://gitlab.pg.innopolis.university/al.petrenko/capstone-judgement-models/-/issues/5

Issue 6:

https://gitlab.pg.innopolis.university/al.petrenko/capstone-judgement-models/-/issues/6

Relevant Merge/Pull Requests

MR for API:

https://gitlab.pg.innopolis.university/al.petrenko/capstone-judgement-models/-/merge_requests/2

MR for Frontend:

https://gitlab.pg.innopolis.university/al.petrenko/capstone-judgement-models/-/merge_requests/1

Model training code

https://gitlab.pg.innopolis.university/al.petrenko/capstone-judgement-models/-/blob/main/init_model/update_model.py

Dataset description

https://gitlab.pg.innopolis.university/al.petrenko/capstone-judgement-models/-/blob/main/structure_of_dataset.md

Experimentation setup

https://gitlab.pg.innopolis.university/al.petrenko/capstone-judgement-models/-/blob/main/diagram_description.md

API Documentation

http://10.93.27.9:8000/docs

Demo

https://disk.yandex.ru/d/e7p5W8XqYowruQ

Demo transcript

For the frontend, we use Streamlit. The current interface includes the main application pages for managing judgement models and running experiments. While the underlying experimental logic will be implemented in future milestones, the frontend structure is already in place and connected to the deployed environment, but may be changed slightly. Streamlit is running on port 8501 of our server.

We established the initial architecture of the project. The backend is implemented using FastAPI and serves as the central communication layer between the user interface and future system components.

On the screen, you can see the automatically generated Swagger documentation provided by FastAPI, which is accessible by port 8000/docs. It allows developers to inspect available endpoints, view request and response formats, and test API functionality directly from the browser.

At this stage, the backend skeleton, API routes, request and response schemas, and the overall project structure have been set up. Some routes may change during the project, if we find the better implementation of our project.

In addition, the university-provided virtual machine was configured and prepared for development and deployment. The project environment was set up, dependencies were installed, and both FastAPI and Streamlit applications were deployed. systemctl (читать как систем си ти эль) was used to ensure that the applications continue running independently of active SSH sessions.

Next Steps

During the next project iteration (Week 3), the team will focus on extending the initial project skeleton and preparing the first end-to-end experiments.

The planned activities include:

- Continue development of the FastAPI backend skeleton and core API endpoints.
- Continue development of the Streamlit frontend skeleton and improve integration with the backend.
- Investigate binary criterion vector conversion rules and explore approaches for mapping generated criteria to the reference evaluation model.
- Define the integration contract with the existing feedback-quality benchmark.
- Prepare the generated evaluation models for benchmark evaluation.
- Conduct the first benchmark experiments and collect initial baseline measurements.
- Analyze the obtained results and identify potential improvements for the evaluation model generation pipeline.
- Prepare and submit the Week 3 progress report.

Special attention will be given to the evaluation-model vector conversion problem, as it represents one of the main technical risks of the project and directly affects the correctness of benchmark evaluation.